SINGLE

@singletecnologia



◆ Case real: Power BI + SQL Server

De 7h35 para 30 minutos

Mesmas regras de negócio, mesmos números. Só o caminho mudou.

Arraste →

O cenário

Um dashboard que virou gargalo

Um dashboard de Clientes no Power BI, alimentado por SQL nativo e com várias mesclagens no Power Query.

Na madrugada
média, com pico de 3h25

1h48

Durante o dia
no horário de pico

7h35

No Desktop
a query principal nem terminava

100+ min

Atualizando conjunto de dados...
8h08 em execução

Falhou

O diagnóstico

A query fazia trabalho desnecessário

- **Processava o histórico inteiro**
  e só no final descartava o que não seria usado
- **Lia a mesma tabela 3 vezes**
- **Repetia a mesma agregação pesada 2 vezes**
- **Fazia as junções na memória**
  as mesclagens do Power Query obrigavam a baixar tudo antes

O que mudou

6 ajustes, zero regra alterada

1. **Filtrar o período na primeira leitura**
   e não no fim da query
2. **Ler cada tabela uma vez só**
   em tabelas temporárias (#temp): no SQL Server, a CTE é recalculada a cada uso
3. **DATEFROMPARTS no lugar de FORMAT**
   o mesmo primeiro dia do mês, com muito menos custo
4. **Função de janela no lugar de auto-junção**
5. **Uma agregação no lugar de duas**
   separando os meses com CASE
6. **Relacionamentos no modelo**
   no lugar das mesclagens do Power Query

Na prática

Antes e depois, no SQL

exemplo-ficticio.sql

```sql
-- exemplo com nomes fictícios
-- 1 leitura só, já filtrada
SELECT ClienteID, Data, Valor
INTO #pedidos
FROM Pedidos
WHERE Data >= @Inicio;

-- antes: FORMAT(Data, 'yyyy-MM-01')
DATEFROMPARTS(YEAR(Data), MONTH(Data), 1)

-- antes: a tabela juntada com ela mesma
MAX(Data) OVER (PARTITION BY ClienteID)
```

- Para medir cada etapa: **SET STATISTICS TIME, IO ON** no SSMS. Ele mostra onde o tempo vai e se a query está calculando ou só esperando.

◆ O resultado

Query principal

~~100+ min~~ **9 min**

Atualização de dia

~~7h35~~ **~30 min**

Números validados antes de ir para produção

100%

● Mesmas regras de negócio

Performance raramente pede uma máquina maior. Quase sempre pede menos trabalho desnecessário.

Salve para o próximo BI lento